

University of Science and Technology — Zewail City

School of Engineering

CIE 555: Neural Networks and Deep Learning

Face Verification & Identification

Siamese Networks on the LFW Dataset

Author

Ahmed Gamal ID: 202200133

Version

V1 — Initial Implementation

Note: This report documents the first version of the project. A second version (V2) is currently in progress, incorporating MTCNN face detection preprocessing, ResNet50V2 and InceptionResNetV2 backbones, and semi-hard triplet mining.

Spring 2026

Contents

1	Overview	2
1.1	Objective	2
1.2	Assignment Requirements Addressed	2
2	Dataset	2
2.1	Source	2
2.2	Filtering Rationale	3
2.3	Identity Distribution	3
2.4	Sample Identities	3
2.5	Train / Validation Split	3
3	Data Pipeline	4
3.1	Pair Generator	4
3.2	Triplet Generator	4
4	Loss Functions	4
4.1	Contrastive Loss	4
4.2	Triplet Loss	4
4.3	Euclidean Distance Layer	5
5	Model Architectures	5
5.1	Architecture 1 — Custom CNN Backbone	5
5.2	Architecture 2 — EfficientNetB0 (Transfer Learning)	5
6	Training Configuration	6
7	Evaluation	6
7.1	Metrics	6
7.2	ROC Curves	7
7.3	Distance Distributions	9
7.4	Threshold Calibration	10
7.5	Summary of Results	11
7.6	Inference Examples	11
8	Discussion	12
8.1	Why did the Custom CNN outperform EfficientNetB0 on Contrastive Loss?	12
8.2	Why did Triplet Loss underperform Contrastive Loss in V1?	12
8.3	Known Limitations (V1)	12
9	Planned Improvements (V2)	12
10	Conclusion	13

Overview

Face verification is the task of determining whether two images depict the same individual. Unlike face recognition — which assigns an identity label from a closed set — verification is a binary decision that must generalize to identities never seen during training. This distinction makes it significantly harder and more realistic.

This report documents the first version of a face verification pipeline built using **Siamese Networks** on the Labeled Faces in the Wild (LFW) dataset. The pipeline implements and compares two backbone architectures across two loss functions, forming a 2×2 experimental grid.

Objective

Given two face images, determine whether they belong to the same person.

Assignment Requirements Addressed

- Custom CNN backbone designed from scratch
- Transfer learning backbone (EfficientNetB0)
- Contrastive Loss implemented manually
- Triplet Loss implemented manually
- Rigorous evaluation: AUC-ROC, distance distributions, threshold calibration
- Inference pipeline with visual output

Dataset

Source

The **Labeled Faces in the Wild (LFW)** dataset is a benchmark database of face photographs collected from the web, designed for studying unconstrained face recognition. It contains 13,233 images of 5,749 individuals.

Table 1: Dataset Properties

Property	Details
Source	Kaggle / University of Massachusetts
Total identities	5,749
Total images	13,233
Filter applied	≥ 12 images per identity
Identities retained	~ 130
Image size used	$105 \times 105 \times 3$

Filtering Rationale

Only identities with at least 12 images were retained. Identities with fewer images produce only trivially easy positive pairs (same lighting and pose across few shots), which causes the network to overfit on superficial cues rather than learning genuine facial similarity.

Identity Distribution

Figure 1 shows the image count per identity after filtering. A severe class imbalance is visible — a small number of identities (e.g. George W. Bush with over 500 images) dominate the dataset. This imbalance is a known limitation of V1 and will be addressed in V2 through per-identity image capping.

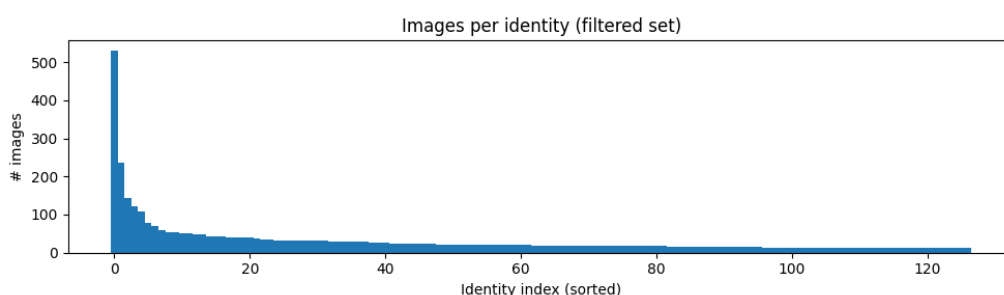


Figure 1: Images per identity after filtering (≥ 12 images). The distribution is heavily right-skewed, with a few identities contributing disproportionately to the training pairs.

Sample Identities

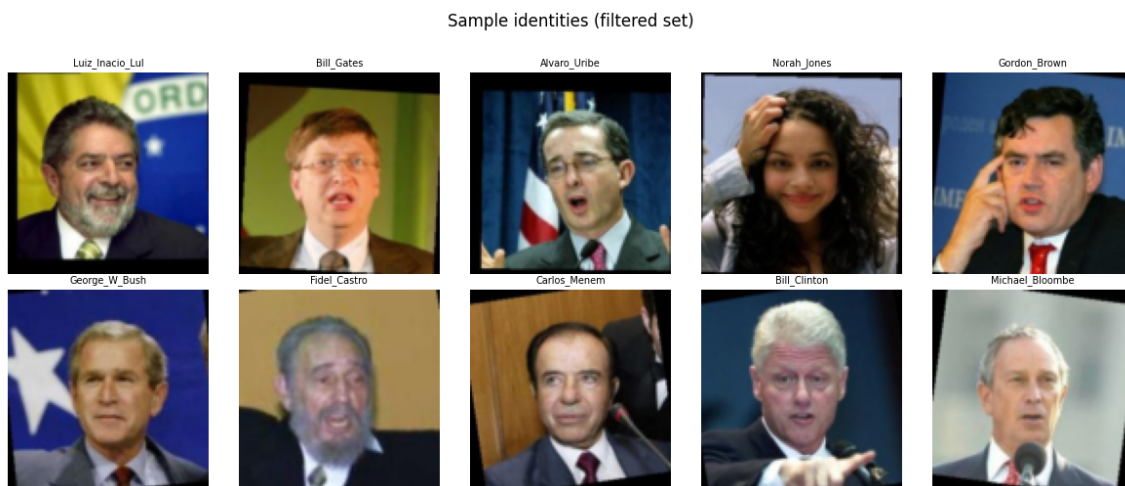


Figure 2: Sample identities from the filtered LFW set. Images vary in pose, lighting, expression, and image quality — reflecting the unconstrained nature of the dataset.

Train / Validation Split

An **identity-level** 80/20 split was applied. This ensures that the validation set contains completely unseen individuals, producing a harder and more realistic evaluation scenario.

An image-level split would allow the model to see the same person in both training and validation, which would artificially inflate performance.

Data Pipeline

Pair Generator

For the Contrastive Loss experiments, image pairs were generated as follows:

- **Positive pairs** (label = 1): all combinations of images from the same identity, sampled up to `n_pairs_per_person = 20` for training.
- **Negative pairs** (label = 0): random cross-identity pairs, with `neg_ratio = 1.0` ensuring a 50/50 label balance.

Triplet Generator

For the Triplet Loss experiments, triplets (A, P, N) were generated randomly:

- **Anchor** and **Positive**: two different images of the same identity.
- **Negative**: one image of a randomly chosen different identity.
- `n_triplets_per_person = 25` for training, 15 for validation.

Note: V2 will replace random triplet sampling with semi-hard triplet mining, selecting hard positives (maximum intra-class distance) and semi-hard negatives (margin-violating but not collapse-inducing).

Loss Functions

Both loss functions were implemented manually from scratch.

Contrastive Loss

$$\mathcal{L}_{\text{contrastive}} = y \cdot d^2 + (1 - y) \cdot \max(m - d, 0)^2$$

where d is the Euclidean distance between embeddings, $y \in \{0, 1\}$ is the pair label, and $m = 1.0$ is the margin. Same-person pairs are penalized for large distances; different-person pairs are penalized when their distance is smaller than the margin.

Triplet Loss

$$\mathcal{L}_{\text{triplet}} = \max(d(A, P) - d(A, N) + m, 0)$$

where $d(A, P)$ is the anchor-positive distance, $d(A, N)$ is the anchor-negative distance, and $m = 0.4$ is the margin. The loss enforces that positive pairs are always closer than negative pairs by at least the margin.

Euclidean Distance Layer

Both models use a shared Euclidean distance computation:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_i (x_i - y_i)^2}$$

implemented as a Keras Lambda layer to ensure it participates in the computation graph.

Model Architectures

Architecture 1 — Custom CNN Backbone

A 4-block convolutional network designed from scratch for face similarity.

Table 2: Custom CNN Architecture

Block	Layers	Design Rationale
Block 1	Conv2D(32) ×2, BN, MaxPool, Dropout(0.15)	Low-level edges and textures
Block 2	Conv2D(64) ×2, BN, MaxPool, Dropout(0.20)	Mid-level shapes and contours
Block 3	Conv2D(128) ×2, BN, MaxPool, Dropout(0.25)	Semantic facial features
Block 4	Conv2D(256), BN, GlobalAvgPool	High-level identity features
Head	Dense(256), Dropout(0.30), Dense(256), L2-norm	Compact embedding

Key design choices:

- **Progressive filter doubling (32→256)**: allows the network to build hierarchical representations from low-level to identity-discriminative features.
- **BatchNormalization after every conv**: stabilizes training and reduces internal covariate shift.
- **Progressive Dropout**: increasing regularization deeper in the network where overfitting is most likely.
- **GlobalAveragePooling instead of Flatten**: reduces parameter count and spatial overfitting.
- **L2-normalized output**: embeddings lie on a unit hypersphere, making the margin in contrastive/triplet loss geometrically consistent across batches.
- **embedding_dim = 256**: compact but expressive enough for the size of the filtered dataset.

Architecture 2 — EfficientNetB0 (Transfer Learning)

EfficientNetB0 pretrained on ImageNet was used as a frozen feature extractor with a trainable embedding head.

Why EfficientNetB0?

- Compounds width, depth, and resolution scaling simultaneously — best accuracy-per-parameter ratio among standard backbones.
- B0 variant ($\sim 5.3\text{M}$ parameters) runs efficiently on Colab T4.
- ImageNet pretraining transfers well: low-level features (edges, textures, shapes) are directly applicable to facial feature extraction.

Training strategy:

- **Phase 1 (warm-up):** backbone fully frozen, only the embedding head trains. Prevents random head weights from corrupting pretrained features early in training.
- **Phase 2 (fine-tuning):** layers from index 150 onward are unfrozen (~ 30 top layers), allowing the backbone to adapt high-level features to the facial similarity task. Lower layers retain general ImageNet features.

Input images are rescaled from $[0, 1]$ to $[0, 255]$ before the backbone, as EfficientNet's internal normalization expects this range.

Training Configuration

Table 3: Training Configuration

Parameter	Value
Input size	$105 \times 105 \times 3$
Embedding dimension	256
Batch size (warm-up)	32
Batch size (fine-tuning)	16
Optimizer	Adam
Warm-up LR	1×10^{-3}
Fine-tuning LR	1×10^{-4}
Contrastive margin	1.0
Triplet margin	0.4
Early stopping patience	12 epochs
LR reduction factor	0.4 (patience: 5 epochs)
Seed	42 (Python, NumPy, TensorFlow)

Evaluation

Metrics

- **AUC-ROC:** threshold-independent measure of discrimination ability. The gold standard for verification tasks.
- **Distance distribution:** overlapping histograms of same-person vs different-person Euclidean distances. A well-trained model shows minimal overlap.

- **F1-calibrated threshold:** the decision threshold is not fixed at 0.5 but swept across the range to find the value maximizing F1 score.

ROC Curves

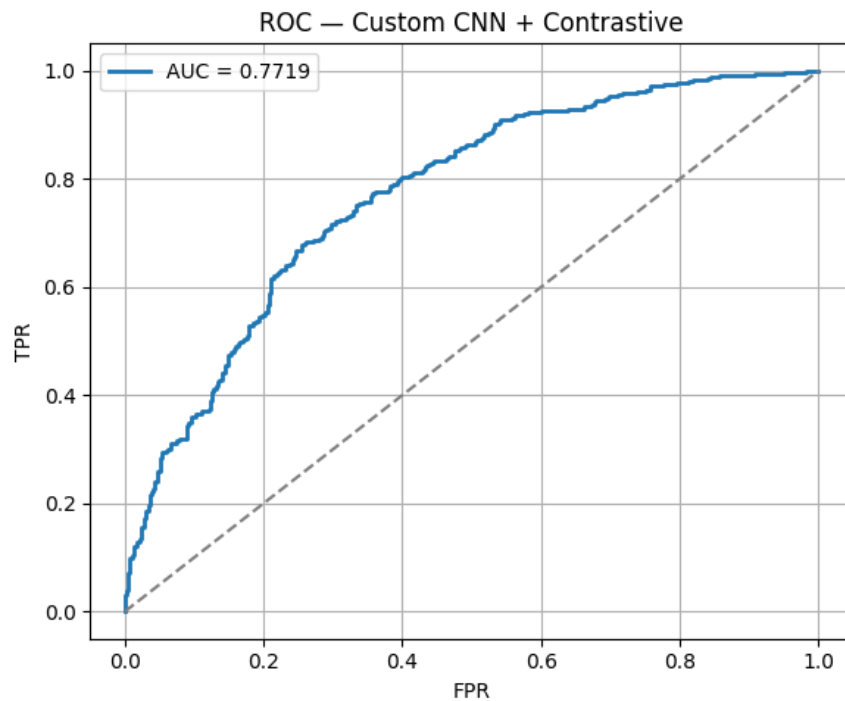


Figure 3: ROC Curve — Custom CNN + Contrastive Loss. AUC = 0.7719 (best result across all four experiments).

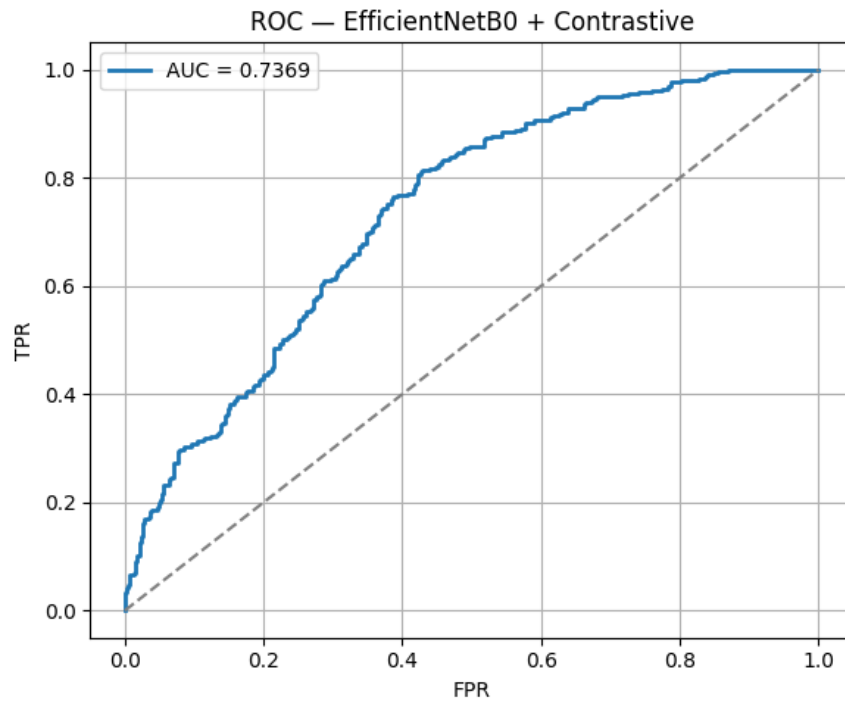


Figure 4: ROC Curve — EfficientNetB0 + Contrastive Loss. AUC = 0.7369.

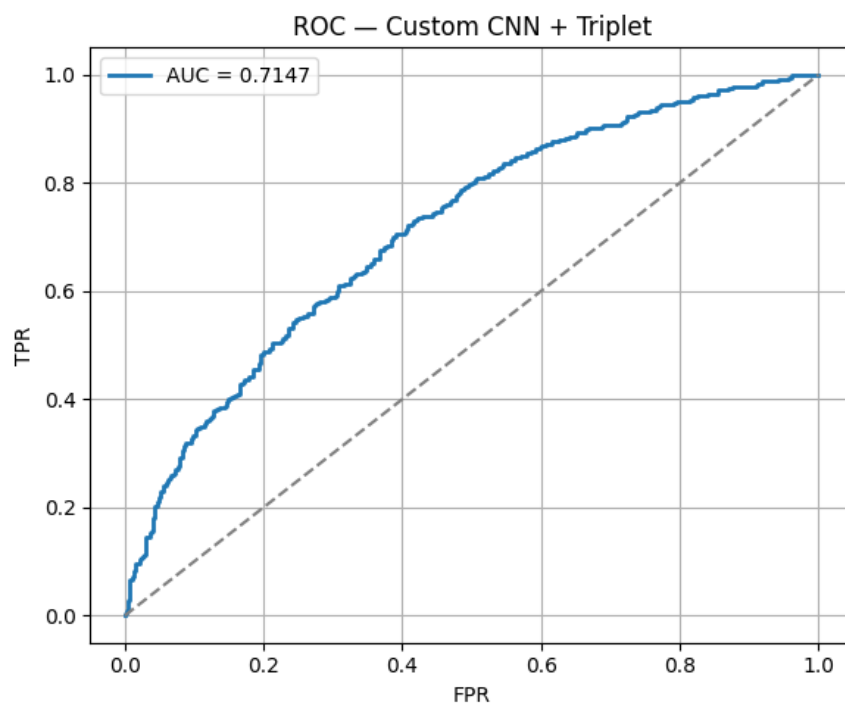


Figure 5: ROC Curve — Custom CNN + Triplet Loss. AUC = 0.7147.

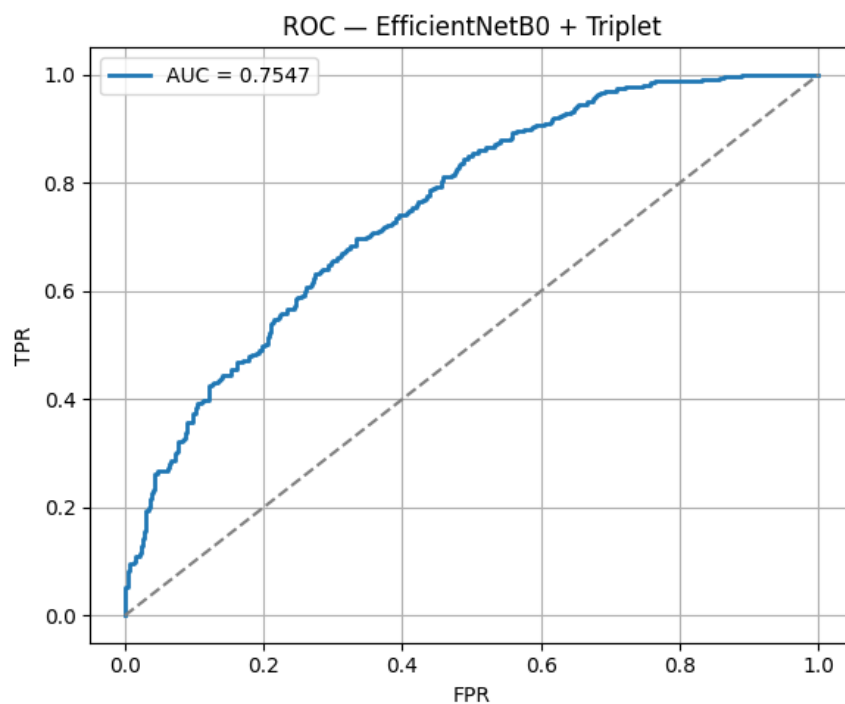


Figure 6: ROC Curve — EfficientNetB0 + Triplet Loss. AUC = 0.7547.

Distance Distributions

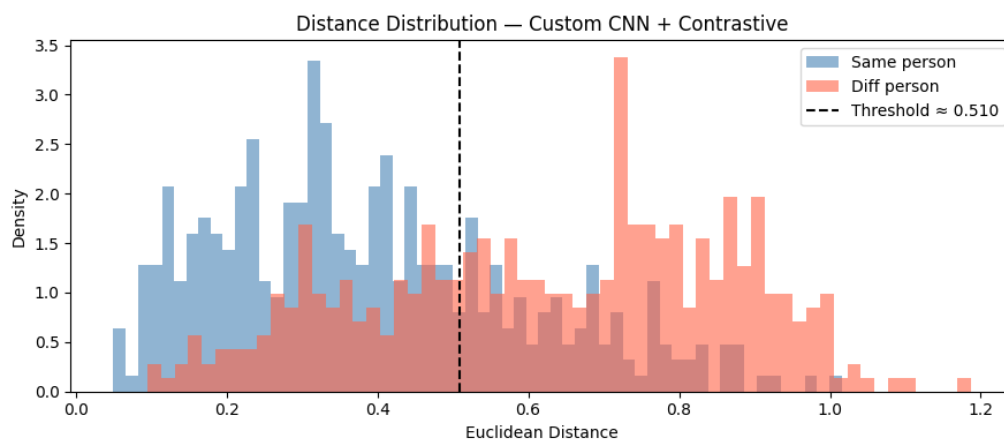


Figure 7: Distance distribution — Custom CNN + Contrastive Loss. Threshold ≈ 0.510 . Some separation is visible between same-person (blue) and different-person (red) distributions, though overlap remains significant in V1.

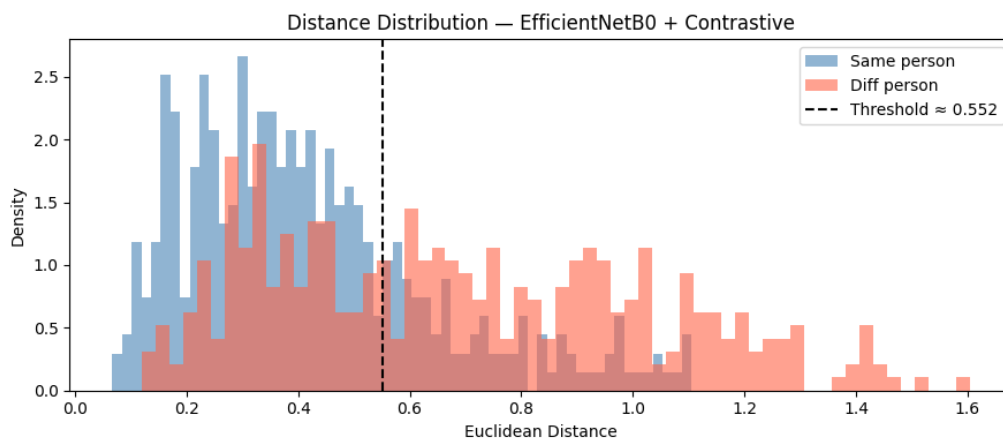


Figure 8: Distance distribution — EfficientNetB0 + Contrastive Loss. Threshold ≈ 0.552 . The different-person distribution has a longer tail, suggesting EfficientNetB0 pushes negative pairs further apart more consistently.

Threshold Calibration

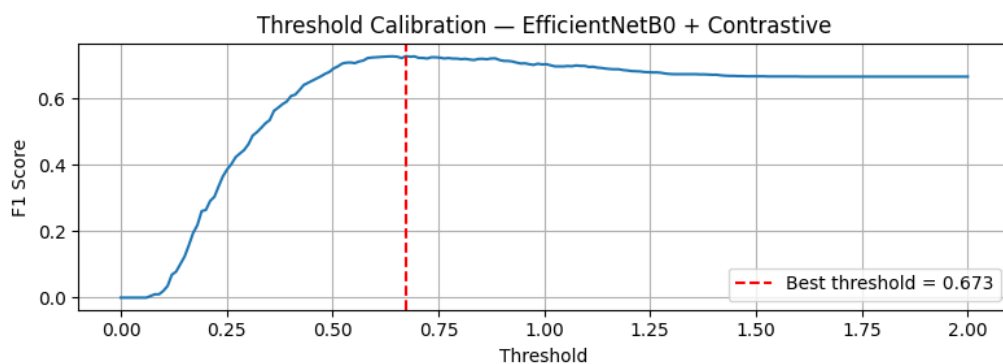


Figure 9: F1-calibrated threshold sweep — EfficientNetB0 + Contrastive Loss. Optimal threshold = 0.673, maximizing F1 score.

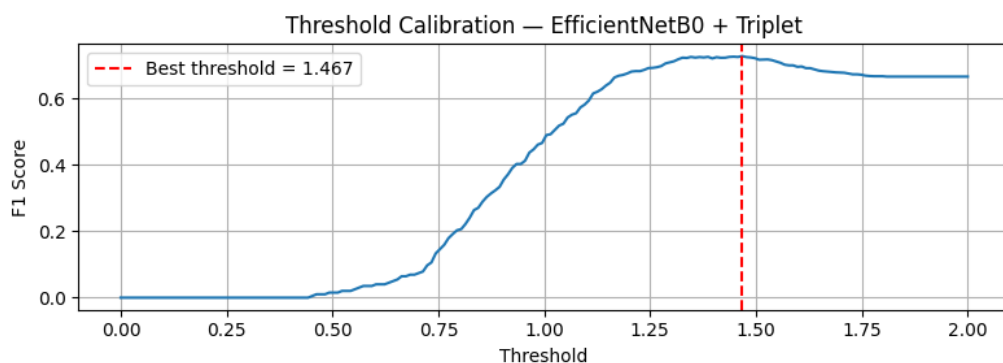


Figure 10: F1-calibrated threshold sweep — EfficientNetB0 + Triplet Loss. Optimal threshold = 1.467, reflecting the larger distance scale produced by Triplet Loss embeddings.

Summary of Results

Table 4: Final Results Comparison

Backbone	Loss	AUC-ROC	Optimal Threshold
Custom CNN	Contrastive	0.7719	0.510
EfficientNetB0	Triplet	0.7547	1.467
EfficientNetB0	Contrastive	0.7369	0.673
Custom CNN	Triplet	0.7147	—

Inference Examples

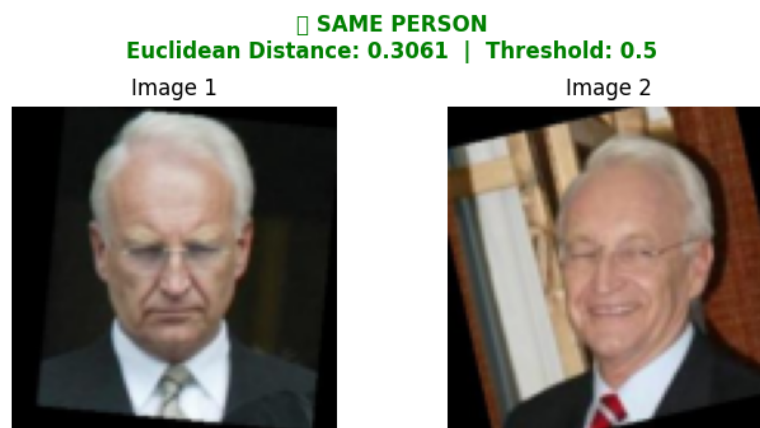


Figure 11: Inference example 1 — same person correctly identified. Euclidean distance = 0.3061, below threshold 0.5.

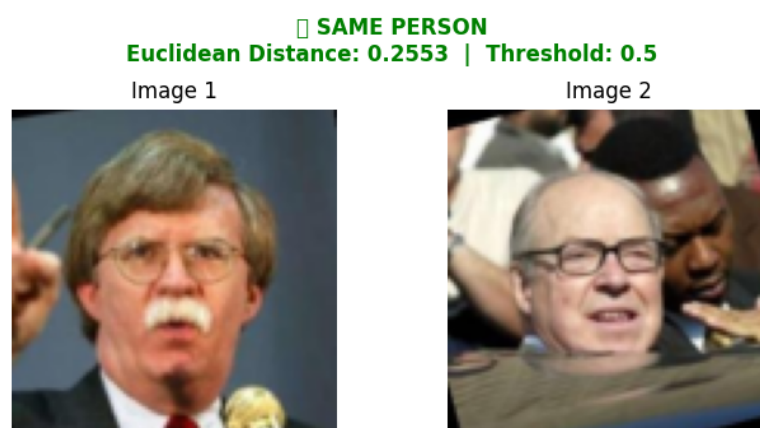


Figure 12: Inference example 2 — same person correctly identified. Euclidean distance = 0.2553, below threshold 0.5.

Discussion

Why did the Custom CNN outperform EfficientNetB0 on Contrastive Loss?

The custom CNN achieved the highest AUC (0.7719) despite having no pretrained weights. The most likely explanation is the **class imbalance problem**: EfficientNetB0's pretrained features were calibrated to a balanced ImageNet distribution, and fine-tuning on a heavily skewed LFW subset (a few identities with 500+ images) pulls the embeddings toward the dominant identities. The custom CNN, trained entirely from scratch on the same skewed data, implicitly adapts to the imbalance. This is a known failure mode of transfer learning on imbalanced datasets.

Why did Triplet Loss underperform Contrastive Loss in V1?

Triplet Loss with **random** triplet sampling is highly inefficient: most randomly sampled triplets are trivially easy (the anchor-negative distance is already large enough to satisfy the margin), contributing zero gradient. The result is slow convergence and weaker representations. V2 will address this with semi-hard triplet mining, which is the key innovation in the FaceNet paper and significantly improves triplet-based training.

Known Limitations (V1)

1. **No face detection**: full 250×250 images fed to the model, including background, hair, and clothing. The model may learn non-facial cues.
2. **Class imbalance**: no per-identity image cap, so a few celebrities dominate pair generation.
3. **Random triplet sampling**: no hard mining, leading to slow triplet loss convergence.
4. **Fixed backbones**: only Custom CNN and EfficientNetB0 evaluated. V2 will add ResNet50V2 and InceptionResNetV2.

Planned Improvements (V2)

- **MTCNN face detection**: automatically detect and crop the facial region with a 15% margin, eliminating background noise.
- **Identity-level image cap** (≤ 30 images per identity): fix class imbalance.
- **ResNet50V2**: DeepFace-style backbone for pair verification with Binary Cross-Entropy.
- **InceptionResNetV2**: FaceNet-style backbone for triplet verification.
- **Semi-hard triplet mining**: select hard positives and semi-hard negatives to eliminate trivial triplets and accelerate convergence.
- **Image size upgrade**: $105 \times 105 \rightarrow 160 \times 160$ (FaceNet standard).

Conclusion

This report presents the first version of a face verification pipeline using Siamese Networks on the LFW dataset. Four experiments were conducted across a 2×2 grid of backbone architectures and loss functions. The best result was achieved by the Custom CNN with Contrastive Loss (AUC-ROC = 0.7719), which counterintuitively outperformed the pretrained EfficientNetB0 backbone — an outcome attributed to the heavy class imbalance in the filtered dataset.

The results are competitive for a V1 implementation with no face detection preprocessing and random triplet sampling. The planned improvements in V2 — particularly MTCNN cropping, identity capping, and semi-hard triplet mining — are expected to significantly improve all metrics.